

High-Performance Rendering for 3D Gaussian Splatting: A Deep Dive into Custom CUDA Kernel Implementation

Section 1: Mathematical Foundations of the Differentiable Rasterizer

The remarkable real-time performance of 3D Gaussian Splatting (3DGS) is predicated on a rendering pipeline that is not only highly optimized for modern GPU architectures but also fully differentiable, enabling gradient-based optimization of the scene representation.⁸ Before delving into the intricacies of the CUDA implementation, it is imperative to establish the mathematical framework that underpins the entire process. This section derives the necessary equations for representing, projecting, and compositing the 3D Gaussian primitives that form the basis of the scene.

1.1 Representing the Scene with 3D Gaussians

Unlike traditional mesh-based rasterization, which uses triangles, or volumetric methods like Neural Radiance Fields (NeRFs), which use MLPs, 3DGS represents a scene as a large collection of anisotropic 3D Gaussians.¹² Each Gaussian describes a localized, continuous distribution of properties within the 3D space.

A 3D Gaussian function, centered at a mean position μ , is defined by its covariance matrix Σ and evaluates the density at any point $p \in \mathbb{R}^3$ as:

$$G(p) = \exp(-\frac{1}{2}(p-\mu)^T \Sigma^{-1}(p-\mu))$$

This function forms the geometric primitive. To render a scene, each Gaussian is augmented with additional properties that are optimized to match captured images of the scene. The

complete set of learnable parameters for a single Gaussian is as follows ¹⁶:

- **Mean (μ):** A 3D vector $([x, y, z])$ representing the center of the Gaussian in world space. This is initialized from the sparse point cloud generated by Structure-from-Motion (SfM) techniques. ¹⁵
- **Covariance (Σ):** A 3×3 symmetric positive semi-definite matrix that defines the shape and orientation of the Gaussian ellipsoid. Direct optimization of a 3×3 matrix is challenging due to the need to maintain its properties. Therefore, the covariance is decomposed into two more intuitive and numerically stable components:
 - A 3D **scaling vector** $s = [s_x, s_y, s_z]$, which defines the extent of the ellipsoid along its principal axes.
 - A rotation quaternion $q = [q_w, q_x, q_y, q_z]$, which defines the orientation of the ellipsoid in space.

The rotation quaternion is converted to a 3×3 rotation matrix R , and the scaling vector is used to form a diagonal scaling matrix S . The full covariance matrix is then reconstructed as $\Sigma = R S S^T R^T$. This decomposition is crucial because it transforms a constrained matrix optimization problem into an unconstrained optimization over the elements of q and s , which is more amenable to standard stochastic gradient descent (SGD) optimizers. The scaling factors are typically passed through an activation function like exp or softplus to ensure they remain positive.
- **Opacity (α):** A scalar value representing the opacity of the Gaussian. This parameter is passed through a sigmoid activation function, $\sigma(\alpha_{\text{param}})$, to ensure the final opacity value lies within the range $[0, 1]$.
- **Color (c):** To model view-dependent effects (e.g., specular reflections), the color is not represented by a simple RGB triplet. Instead, it is encoded using **Spherical Harmonics (SH)** coefficients. ¹³ SH are a basis of functions defined on the surface of a sphere, analogous to a Fourier series. By storing a set of SH coefficients for each color channel, the color can be evaluated for any given viewing direction, allowing the appearance of the Gaussian to change depending on the camera's position. The number of coefficients determines the complexity of the view-dependent effects that can be captured, with a degree of 3 (16 coefficients per color channel) being a common choice. ¹³

The following table provides a concise summary of the parameterization used for each Gaussian, which directly informs the data structures required for a GPU implementation.

Parameter	Mathematical Symbol	Representation	Number of Floats	Description
Position	μ	3D Vector	3	The (x, y, z) center of the Gaussian in

				world space.
Rotation	q	Quaternion (wxyz)	4	Represents the 3D orientation of the Gaussian ellipsoid.
Scale	s	3D Vector	3	Represents the scaling along the principal axes of the ellipsoid.
Opacity	α	Scalar	1	The learnable parameter for opacity (pre-sigmoid).
Color (SH)	c_{sh}	SH Coefficients	$3 \times (\text{degree} + 1)^2$	Coefficients for modeling view-dependent color. For degree 3, this is 48 floats.

1.2 Projecting Anisotropic 3D Gaussians to 2D

To render an image, the 3D Gaussians must be projected onto the 2D image plane of a virtual camera. While projecting the 3D mean μ is straightforward using standard camera transformation matrices, projecting the 3D covariance Σ is more complex. The original 3DGS paper employs a first-order Taylor approximation of the perspective projection, which is both efficient and accurate for Gaussians that are small relative to their distance from the camera.¹²

Let the camera viewing transformation be given by the matrix W (a 3×4 world-to-camera matrix) and the full perspective projection be P . For a point t in camera space, the

projection onto the image plane (in pixels) is a non-linear function. The core idea is to approximate this projection locally with an affine transformation defined by its Jacobian matrix J . The Jacobian for the perspective projection of a point $p_c = [x, y, z]^T$ in camera space is:

$$J = \begin{bmatrix} f_x/z & 0 & f_y/z - fx/z^2 - fy/z^2 \end{bmatrix}$$

where f_x and f_y are the camera's focal lengths in pixels. The 3D covariance matrix Σ is first transformed into camera space, and then this Jacobian is used to project it into a 2D covariance matrix Σ' in screen space¹⁸:

$$\Sigma' = J \Sigma J^T$$

This 2×2 matrix Σ' describes the shape and orientation of the elliptical splat on the 2D image plane. This projection is a critical step performed for every Gaussian in every frame and is a primary target for CUDA optimization.

1.3 The Alpha Blending Compositing Formula

After projecting all Gaussians into 2D splats, the final color for each pixel is determined by compositing them in order of depth. The 3DGS rasterizer uses a front-to-back alpha blending formula, which is structurally identical to the compositing equation used in volumetric rendering methods like NeRF.⁸ This correspondence is a key reason why 3DGS can be optimized effectively with similar loss functions (e.g., L1 and SSIM).

For a pixel, consider a set of N projected Gaussians that overlap it, sorted by their depth from nearest (1) to farthest (N). The final color C is computed by the sum:

$$C = \sum_{i=1}^N c_i \alpha_i' = \sum_{i=1}^N c_i \prod_{j=1}^{i-1} (1 - \alpha_j')$$

Here, c_i is the color of the i -th Gaussian (evaluated from its SH coefficients using the current view direction), and α_i' is its effective opacity at the pixel center. This effective opacity is the product of the Gaussian's learned opacity α_i and the value of its projected 2D Gaussian function evaluated at the pixel's coordinates:

$$\alpha_i' = \alpha_i \exp(-21(x_{\text{pixel}} - \mu_{2D,i})^T (\Sigma_i')^{-1} (x_{\text{pixel}} - \mu_{2D,i}))$$

The term $\prod_{j=1}^{i-1} (1 - \alpha_j')$ represents the accumulated transmittance, which is the amount of light that has passed through the first $i-1$ Gaussians to reach the i -th one. This front-to-back formulation is highly parallelizable on a per-pixel basis once the Gaussians have been sorted by depth. The entire rendering process—projection, sorting, and blending—is designed to be differentiable, allowing gradients from a loss function computed on the rendered image to flow back to all the learnable parameters of the 3D Gaussians.

Section 2: Architecture of a Tile-Based GPU Rasterizer

Transitioning from the mathematical foundations to a high-performance implementation requires an architecture that maps efficiently to the massively parallel nature of modern GPUs. A naive approach, such as iterating through every pixel and then through every Gaussian for each pixel, would be computationally infeasible due to redundant calculations and inefficient memory access patterns. The 3DGS method instead employs a tile-based rasterizer, a well-established technique in computer graphics that is exceptionally well-suited to this problem.¹⁵

2.1 The Rationale for Tile-Based Rasterization

The core principle of tile-based rasterization is to divide the screen into a grid of small, rectangular tiles (e.g., 16×16 or 32×32 pixels). The rendering workload is then partitioned on a per-tile basis rather than a per-pixel basis. This strategy offers several profound advantages for GPU execution:

1. **Memory Locality and Data Reuse:** By processing a tile at a time, all the Gaussians that affect that tile can be loaded from global GPU memory into faster, on-chip shared memory. A single CUDA thread block can be assigned to a tile, and the threads within that block can then share the data for the Gaussians, significantly reducing the number of expensive global memory reads.¹⁹
2. **Parallelism:** The rendering of each tile is an independent problem. This allows for embarrassingly parallel execution, where each tile can be rendered concurrently by a different thread block on the GPU's streaming multiprocessors (SMs) without requiring synchronization between them.
3. **Work Distribution:** The problem is inverted from a "gather" operation (for each pixel, find all relevant Gaussians) to a "scatter-gather" approach. First, in a "binning" or "scattering" phase, each Gaussian determines which tiles it overlaps. Then, in the rendering phase, each tile "gathers" the list of Gaussians assigned to it and processes them. This structure is a much better fit for the GPU's single-instruction, multiple-data (SIMD) execution model.

This architectural choice is the cornerstone of achieving real-time frame rates. It transforms the rendering of millions of primitives from a memory-bandwidth-bound problem into a compute-bound problem that can be effectively parallelized.

2.2 The Forward Pass Pipeline (Rendering)

The forward pass, which generates a 2D image from the 3D Gaussian scene, is a multi-stage pipeline where each stage is implemented as one or more CUDA kernel launches. The overall data flow is as follows:

1. **Frustum Culling:** The pipeline begins with a lightweight culling stage. A kernel is launched with one thread per Gaussian. Each thread checks if its Gaussian is completely outside the camera's view frustum. Gaussians that are culled are flagged and ignored in subsequent stages. This is a highly effective first-pass optimization that can eliminate a significant portion of the Gaussians from further processing, especially in large scenes.
2. **Projection and Bounding Box Calculation:** For all non-culled Gaussians, a second kernel projects them from 3D world space to 2D screen space. As derived in Section 1.2, this involves calculating the 2D mean μ_{2D} and the 2D covariance matrix Σ . From Σ , the kernel computes the ellipse's axes and determines a conservative 2D bounding box that fully encloses the splat. This bounding box is expressed in tile coordinates (e.g., which range of tiles the splat covers horizontally and vertically).
3. **Binning and Sorting:** This is the architectural lynchpin of the entire rasterizer. The goal is to create a single, globally sorted list of all Gaussian-tile intersections, ordered first by tile ID and then by depth within each tile.
 - **Binning:** A kernel iterates over each Gaussian's tile-based bounding box. For each tile the Gaussian overlaps, it generates a key-value pair. The value is simply the ID of the Gaussian.
 - **Key Generation:** The key is a 64-bit integer designed for sorting. The most significant 32 bits encode the tile_id, and the least significant 32 bits encode the Gaussian's depth (its z-coordinate in camera space, converted to an integer). This composite key ensures that when sorted, all entries for tile_id = 0 will appear first, sorted by depth, followed by all entries for tile_id = 1, also sorted by depth, and so on.
 - **Radix Sort:** A single, highly optimized, device-wide radix sort is performed on this list of 64-bit keys (and their associated Gaussian ID values). The original implementation leverages the CUB library's DeviceRadixSort, a state-of-the-art parallel sorting algorithm for CUDA.¹⁰ This step is often the primary performance bottleneck of the forward pass, but it is what enables the final rendering stage to be so efficient.
4. **Rendering:** The final stage launches one CUDA thread block per screen tile. Each thread block is responsible for rendering its assigned 16×16 pixel area. It uses the globally sorted list to identify the range of Gaussians that belong to its tile. It then cooperatively loads these Gaussians' data into shared memory and iterates through them in sorted (front-to-back) order, applying the alpha blending formula from Section 1.3 for each pixel

in the tile. The final computed colors are written to the output image buffer in global memory.

2.3 The Backward Pass Pipeline (Gradient Computation)

For the model to be trainable, the entire rendering pipeline must be differentiable. The backward pass computes the gradients of a scalar loss function (e.g., the difference between the rendered image and a ground-truth image) with respect to every learnable parameter of every Gaussian. The data flow is essentially the reverse of the forward pass, relying on the chain rule of calculus.

1. **Initialize Gradients:** The process starts with the gradient of the loss with respect to the final rendered image, $\frac{\partial L}{\partial C}$. This is typically a tensor of the same size as the image, where each element is the gradient for the corresponding pixel color.
2. **Backward Rendering:** A kernel, again launched with one thread block per tile, processes the sorted list of Gaussians. However, this time it traverses the list in **reverse order** (back-to-front). This is critical for correctly accumulating gradients through the sequential alpha blending formula. This kernel computes the gradients of the loss with respect to the intermediate 2D splat properties for each Gaussian: $\frac{\partial L}{\partial c_i}$, $\frac{\partial L}{\partial \alpha^i}$, and from that, the gradients with respect to the 2D mean and covariance, $\frac{\partial L}{\partial \mu^{2D, i}}$ and $\frac{\partial L}{\partial \Sigma^i}$. These gradients are accumulated for each Gaussian from all the pixels it affects.
3. **Gradient Backpropagation:** A final kernel is launched, with one thread per Gaussian. Each thread takes the accumulated 2D gradients for its Gaussian and backpropagates them through the projection equations (the reverse of the process in Section 1.2). This involves applying the chain rule with the transpose of the Jacobian matrices to compute the final gradients with respect to the 3D learnable parameters: $\frac{\partial L}{\partial \mu}$, $\frac{\partial L}{\partial q}$, $\frac{\partial L}{\partial s}$, $\frac{\partial L}{\partial \alpha}$, and $\frac{\partial L}{\partial c_{sh}}$. These final gradient tensors can then be passed directly to a PyTorch optimizer (e.g., Adam) to update the Gaussian parameters.

Section 3: The Forward Pass: From 3D Gaussians to 2D Image (With Code)

This section provides a detailed implementation of the forward rendering pass in CUDA C++.

The code presented is illustrative, focusing on clarity and correctness while reflecting the architecture described in Section 2. It is broken down into the distinct kernel launches that constitute the pipeline.

3.1 Kernel 1: Projection and Bounding Box Calculation

This initial kernel is responsible for transforming the 3D Gaussians into the 2D space of the camera. It computes the 2D mean, the 2D covariance, and a screen-space bounding box for each primitive. It is typically launched with a 1D grid where each thread processes a single Gaussian.

Mathematical Preliminaries:

The 3D covariance Σ is constructed from the rotation quaternion q and scale vector s :

$$R = \text{quat_to_mat}(q)$$

$$S = \text{diag}(s)$$

$$\Sigma = R S S^T R^T$$

The projected 2D covariance Σ' is given by $\Sigma' = J W \Sigma W^T J^T$. For performance, the matrix multiplications are carefully ordered. The term $V = W \Sigma W^T$ is computed first. V is a 3×3 matrix in camera space. Then Σ' is computed from V and the Jacobian J .

C++

```
// CUDA Kernel for projecting 3D Gaussians to 2D splats
// Assumes a 1D grid where blockDim.x * gridDim.x >= num_gaussians
__global__ void project_gaussians_kernel(
    // Input buffers
    const float* __restrict__ means3d,    // [N, 3]
    const float* __restrict__ scales,     // [N, 3]
    const float* __restrict__ quats,     // [N, 4]
    const float* __restrict__ view_matrix, // [4, 4]
    const float* __restrict__ proj_matrix, // [4, 4]

    // Output buffers
    float* means2d,           // [N, 2]
    float* depths,           // [N]
    float* covs2d,           // [N, 3] (upper triangular part: xx, xy, yy)
```

```

int* radii,          // [N]

// Parameters
const int num_gaussians,
const float focal_x,
const float focal_y,
const float tan_fovx,
const float tan_fovy,
const int image_width,
const int image_height)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= num_gaussians) return;

    // --- 1. Load Gaussian data for the current thread ---
    float3 p_world = {means3d[3*idx], means3d[3*idx+1], means3d[3*idx+2]};
    float4 q = {quats[4*idx], quats[4*idx+1], quats[4*idx+2], quats[4*idx+3]};
    float3 s = {scales[3*idx], scales[3*idx+1], scales[3*idx+2]};

    // --- 2. Transform 3D mean to camera space ---
    float4 p_hom = {p_world.x, p_world.y, p_world.z, 1.0f};
    float4 p_cam_hom;
    // p_cam_hom = view_matrix * p_hom
    p_cam_hom.x = view_matrix[0]*p_hom.x + view_matrix[1]*p_hom.y + view_matrix[2]*p_hom.z +
view_matrix[3]*p_hom.w;
    p_cam_hom.y = view_matrix[4]*p_hom.x + view_matrix[5]*p_hom.y +
view_matrix[6]*p_hom.z + view_matrix[7]*p_hom.w;
    p_cam_hom.z = view_matrix[8]*p_hom.x + view_matrix[9]*p_hom.y +
view_matrix[10]*p_hom.z + view_matrix[11]*p_hom.w;

    float p_cam_z = p_cam_hom.z;
    depths[idx] = p_cam_z;

    // Culling: Ignore Gaussians behind the camera's near plane
    if (p_cam_z <= 0.1f) {
        radii[idx] = 0; // Mark as culled
        return;
    }

    // --- 3. Project to screen space (NDC -> pixels) ---
    float4 p_clip = {
        proj_matrix[0]*p_cam_hom.x + proj_matrix[1]*p_cam_hom.y + proj_matrix[2]*p_cam_hom.z,
        proj_matrix[4]*p_cam_hom.x + proj_matrix[5]*p_cam_hom.y +
proj_matrix[6]*p_cam_hom.z,

```

```

    proj_matrix[8]*p_cam_hom.x + proj_matrix[9]*p_cam_hom.y +
proj_matrix[10]*p_cam_hom.z,
    proj_matrix[12]*p_cam_hom.x + proj_matrix[13]*p_cam_hom.y +
proj_matrix[14]*p_cam_hom.z
};
float p_ndc_x = p_clip.x / p_clip.w;
float p_ndc_y = p_clip.y / p_clip.w;

means2d[2*idx] = (p_ndc_x + 1.0f) * image_width * 0.5f;
means2d[2*idx+1] = (p_ndc_y + 1.0f) * image_height * 0.5f;

// --- 4. Compute 3D Covariance Matrix ---
// Build rotation matrix R from quaternion q
float r, i, j, k;
r = q.x; i = q.y; j = q.z; k = q.w;
float3x3 R = {
    {1-2*(j*j+k*k), 2*(i*j-r*k), 2*(i*k+r*j)},
    {2*(i*j+r*k), 1-2*(i*i+k*k), 2*(j*k-r*i)},
    {2*(i*k-r*j), 2*(j*k+r*i), 1-2*(i*i+j*j)}
};

// Build scaling matrix S (as diagonal)
float3x3 S = {{s.x, 0, 0}, {0, s.y, 0}, {0, 0, s.z}};

// Sigma = R * S * S^T * R^T. Since S is diagonal, S*S^T is a diagonal matrix with squared scales
float3x3 Sigma = mat_mul(R, mat_mul(diag_mat_mul(S, S), transpose(R)));

// --- 5. Project Covariance to 2D ---
// W is the rotation part of the view matrix
float3x3 W = {{view_matrix, view_matrix[1], view_matrix[2]},
    {view_matrix[4], view_matrix[5], view_matrix[6]},
    {view_matrix[8], view_matrix[9], view_matrix[10]}};

// V = W * Sigma * W^T
float3x3 V = mat_mul(W, mat_mul(Sigma, transpose(W)));

// Jacobian J
float J_00 = focal_x / p_cam_z;
float J_11 = focal_y / p_cam_z;
float J_02 = - (focal_x * p_cam_hom.x) / (p_cam_z * p_cam_z);
float J_12 = - (focal_y * p_cam_hom.y) / (p_cam_z * p_cam_z);

// Sigma' = J * V * J^T

```

```

float cov_xx = J_00*J_00*V.m + 2*J_00*J_02*V.m[8] + J_02*J_02*V.m[8][8];
float cov_yy = J_11*J_11*V.m[4][4] + 2*J_11*J_12*V.m[4][8] + J_12*J_12*V.m[8][8];
float cov_xy = J_00*J_11*V.m[4] + J_00*J_12*V.m[8] + J_02*J_11*V.m[4][8] +
J_02*J_12*V.m[8][8];

// Add a small epsilon for numerical stability, preventing singular matrices
cov_xx += 0.3f;
cov_yy += 0.3f;

covs2d[3*idx] = cov_xx;
covs2d[3*idx+1] = cov_xy;
covs2d[3*idx+2] = cov_yy;

// --- 6. Compute Bounding Box Radius ---
// Find eigenvalues of the 2D covariance matrix to determine the ellipse size
float det = cov_xx * cov_yy - cov_xy * cov_xy;
if (det == 0.0f) {
    radii[idx] = 0;
    return;
}

float mid = 0.5f * (cov_xx + cov_yy);
float lambda1 = mid + sqrt(max(0.1f, mid*mid - det));

// The radius of the bounding circle is ~3 standard deviations
float radius = ceilf(3.0f * sqrt(max(lambda1, 0.0f)));
radii[idx] = static_cast<int>(radius);
}

```

3.2 Kernel 2: Binning and Key Generation

After projection, the next step is to determine which tiles each Gaussian splat overlaps. This kernel generates a list of (key, value) pairs, where the key encodes both tile ID and depth, and the value is the Gaussian index. This list will be the input to the parallel sort.

C++

```

// CUDA Kernel for binning Gaussians into tiles and generating sort keys
__global__ void binning_kernel(
    // Input buffers
    const float* __restrict__ means2d,    // [N, 2]
    const float* __restrict__ depths,    // [N]
    const int* __restrict__ radii,       // [N]

    // Output buffers
    uint64_t* keys,                     // [M] - M is total number of overlaps
    int* values,                         // [M]
    int* tile_gaussian_counts,          // [num_tiles] - Atomic counter per tile

    // Parameters
    const int num_gaussians,
    const int image_width,
    const int image_height,
    const int tile_size)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= num_gaussians) return;

    int radius = radii[idx];
    if (radius == 0) return; // Skip culled Gaussians

    float2 mean = {means2d[2*idx], means2d[2*idx+1]};

    // Determine tile range for this Gaussian
    int num_tiles_x = (image_width + tile_size - 1) / tile_size;

    int tile_min_x = max(0, static_cast<int>((mean.x - radius) / tile_size));
    int tile_max_x = min(num_tiles_x - 1, static_cast<int>((mean.x + radius) / tile_size));
    int tile_min_y = max(0, static_cast<int>((mean.y - radius) / tile_size));
    int tile_max_y = min((image_height + tile_size - 1) / tile_size - 1, static_cast<int>((mean.y +
radius) / tile_size));

    // Convert depth to a 32-bit integer for sorting.
    // Use type punning via a union to reinterpret float bits as an integer.
    union { float f; uint32_t i; } depth_converter;
    depth_converter.f = depths[idx];
    uint32_t depth_key = depth_converter.i;

    // Iterate over the tiles this Gaussian covers
    for (int y = tile_min_y; y <= tile_max_y; ++y) {

```

```

for (int x = tile_min_x; x <= tile_max_x; ++x) {
    int tile_id = y * num_tiles_x + x;

    // Atomically increment the global counter for this tile to get a unique index
    // This is a critical step to avoid race conditions when multiple Gaussians
    // write to the same tile's section of the key-value list.
    // Note: This requires a pre-pass or a more complex atomic scheme to know the
    // starting offset for each tile. A simpler (but less performant) approach
    // is to use a single global atomic counter.
    // For this example, we assume a pre-computed offset `tile_offsets`.
    // int out_idx = atomicAdd(&tile_gaussian_counts[tile_id], 1);
    // This is complex, so for illustration, we'll use a simplified global atomic.
    // A real implementation uses a two-pass approach (count, then prefix-sum).

    // Simplified: Use a single global counter for total overlaps.
    // int out_idx = atomicAdd(global_overlap_counter, 1);
    // keys[out_idx] = (static_cast<uint64_t>(tile_id) << 32) | depth_key;
    // values[out_idx] = idx;
    // The actual implementation is more involved to ensure contiguous memory writes.
}
}
}

```

Note: The binning kernel shown above is simplified. A production-level implementation uses a two-pass approach. First, a kernel counts the number of tile overlaps for each Gaussian. Then, a parallel prefix sum (scan) is performed on these counts to determine the exact starting memory location for each tile's list of Gaussians. This avoids the use of atomics in the main binning loop, which can be a performance bottleneck.

3.3 Device-Wide Radix Sort

This step is not a custom kernel but a crucial call to a library function. The NVIDIA CUB (CUDA Unblocked) library provides a highly optimized, device-wide radix sort that is ideal for this task.¹⁰

C++

```

// Host-side C++ code to invoke the CUB radix sort
#include <cub/cub.cuh>

void sort_keys(
    void* d_temp_storage,    // Temporary storage buffer
    size_t& temp_storage_bytes, // Size of temp storage
    uint64_t* d_keys,        // Device pointer to keys
    int* d_values,           // Device pointer to values
    int num_overlaps,        // Total number of Gaussian-tile overlaps
    cudaStream_t stream)
{
    // First, determine the required size of the temporary storage buffer
    cub::DeviceRadixSort::SortPairs(
        d_temp_storage, temp_storage_bytes,
        d_keys, d_keys, // Sort keys in-place
        d_values, d_values, // Carry values along
        num_overlaps, 0, 64, stream);

    // Allocate the temporary storage
    // cudaMalloc(&d_temp_storage, temp_storage_bytes);

    // Second, perform the actual sort
    cub::DeviceRadixSort::SortPairs(
        d_temp_storage, temp_storage_bytes,
        d_keys, d_keys,
        d_values, d_values,
        num_overlaps, 0, 64, stream);

    // cudaFree(d_temp_storage);
}

```

3.4 Kernel 3: Tile-Based Rendering

This is the final and most performance-critical kernel of the forward pass. It is launched with a 2D grid where each thread block corresponds to one tile on the screen. Threads within a block cooperate to render the pixels of that tile. Shared memory is used extensively to cache Gaussian data and minimize global memory access.

C++

```
// CUDA Kernel for rendering tiles using the sorted Gaussian list
__global__ void render_kernel(
    // Input buffers
    const uint64_t* __restrict__ keys,      // Sorted keys (tile_id, depth)
    const int* __restrict__ values,        // Sorted Gaussian IDs
    const int* __restrict__ tile_ranges,    // [num_tiles, 2] start/end index for each tile
    const float* __restrict__ means2d,     // [N, 2]
    const float* __restrict__ covs2d,      // [N, 3]
    const float* __restrict__ colors,      // [N, 3] or SH coefficients
    const float* __restrict__ opacities,    // [N]

    // Output buffer
    float* out_image,                     //

    // Parameters
    const int image_width,
    const int image_height,
    const int tile_size)
{
    // --- 1. Identify tile and pixel coordinates ---
    int tile_x = blockIdx.x;
    int tile_y = blockIdx.y;
    int num_tiles_x = (image_width + tile_size - 1) / tile_size;
    int tile_id = tile_y * num_tiles_x + tile_x;

    int thread_idx = threadIdx.x;
    int block_size = blockDim.x; // e.g., 256 for a 16x16 tile

    // Each thread handles a subset of pixels in the tile
    int pix_in_tile_x = thread_idx % tile_size;
    int pix_in_tile_y = thread_idx / tile_size;

    // Global pixel coordinates
    int pix_x = tile_x * tile_size + pix_in_tile_x;
    int pix_y = tile_y * tile_size + pix_in_tile_y;

    // --- 2. Initialize pixel color and transmittance ---
    float3 final_color = {0.0f, 0.0f, 0.0f};
    float T = 1.0f; // Transmittance

    // --- 3. Get the range of Gaussians for this tile ---
```

```

int start_idx = tile_ranges[2 * tile_id];
int end_idx = tile_ranges[2 * tile_id + 1];
int num_gaussians_in_tile = end_idx - start_idx;

// --- 4. Cooperatively load and process Gaussians in batches ---
extern __shared__ float shared_data;
// Layout of shared memory:
// float2* smem_means = (float2*)shared_data;
// float3* smem_covs = (float3*)&smem_means;
// float3* smem_colors= (float3*)&smem_covs;
// float* smem_opacs = (float*)&smem_colors;

for (int i = 0; i < num_gaussians_in_tile; i += block_size) {
    int load_idx = start_idx + i + thread_idx;
    if (load_idx < end_idx) {
        int gaussian_id = values[load_idx];
        // Load data for one Gaussian into shared memory
        // smem_means[thread_idx] =...
        // smem_covs[thread_idx] =...
    }
    __syncthreads();

    // Iterate through the batch of Gaussians loaded in shared memory
    for (int j = 0; j < block_size && (i + j) < num_gaussians_in_tile; ++j) {
        if (T < 1e-4f) break; // Early exit if pixel is opaque

        // --- 5. Evaluate 2D Gaussian and compute alpha ---
        // float2 mean = smem_means[j];
        // float3 cov = smem_covs[j];
        //... (load other data)

        float dx = pix_x - mean.x;
        float dy = pix_y - mean.y;

        float det = cov.x * cov.z - cov.y * cov.y;
        if (det == 0.0f) continue;
        float inv_det = 1.0f / det;

        float inv_cov_xx = cov.z * inv_det;
        float inv_cov_yy = cov.x * inv_det;
        float inv_cov_xy = -cov.y * inv_det;

        float power = -0.5f * (inv_cov_xx*dx*dx + inv_cov_yy*dy*dy + 2.0f*inv_cov_xy*dx*dy);

```

```

    if (power > 0.0f) continue; // Should not happen with positive definite cov

    float G_val = expf(power);
    float alpha_prime = opacity * G_val;

    // --- 6. Alpha Blending ---
    // Note: This is a simplified blending. A more precise implementation
    // would clamp alpha_prime to avoid T becoming negative.
    alpha_prime = min(0.99f, alpha_prime);

    float new_T = T * (1.0f - alpha_prime);
    final_color.x += T * alpha_prime * color.x;
    final_color.y += T * alpha_prime * color.y;
    final_color.z += T * alpha_prime * color.z;
    T = new_T;
}
__syncthreads();
}

// --- 7. Write final color to global memory ---
if (pix_x < image_width && pix_y < image_height) {
    int pixel_offset = (pix_y * image_width + pix_x) * 3;
    out_image[pixel_offset] = final_color.x;
    out_image[pixel_offset + 1] = final_color.y;
    out_image[pixel_offset + 2] = final_color.z;
}
}

```

Section 4: The Backward Pass: Enabling Optimization (With Code)

The differentiability of the rasterizer is what enables the 3D Gaussian representation to be optimized via gradient descent. The backward pass computes the gradients of the loss function with respect to all learnable parameters by applying the chain rule in reverse through the rendering pipeline. This process is more complex than the forward pass due to the intricate data dependencies in the alpha blending formula.

4.1 Mathematical Derivation of Gradients

Let the loss function be L . The process starts with the gradient of the loss with respect to the final pixel color, $\frac{\partial L}{\partial C}$, which is provided by the PyTorch autograd engine. We need to find the gradients with respect to the parameters of each Gaussian i .

The color C is a sum over all Gaussians: $C = \sum_{i=1}^N C_i$, where $C_i = c_i \alpha_i' T_{i-1}$ and $T_{i-1} = \prod_{j=1}^{i-1} (1 - \alpha_j')$.

Using the chain rule, the gradient with respect to the color c_i is straightforward:

$$\frac{\partial L}{\partial c_i} = \frac{\partial L}{\partial C} \frac{\partial C}{\partial c_i} = \frac{\partial L}{\partial C} \alpha_i' T_{i-1}$$

The gradient with respect to the effective alpha α_i' is more complex as it affects all subsequent color contributions C_k for $k > i$:

$$\frac{\partial L}{\partial \alpha_i'} = \frac{\partial L}{\partial C} \left(\frac{\partial C_i}{\partial \alpha_i'} + \sum_{k=i+1}^N \frac{\partial C_k}{\partial \alpha_i'} \right) = \frac{\partial L}{\partial C} (c_i T_{i-1} - \sum_{k=i+1}^N c_k T_{k-1})$$

This backward accumulation is why the backward rendering kernel must process Gaussians in a back-to-front order. Once $\frac{\partial L}{\partial \alpha_i'}$ is known, further application of the chain rule yields gradients for the 2D mean and covariance, which are then backpropagated to the 3D parameters.

4.2 Kernel 4: Backward Rendering and 2D Gradient Computation

This kernel mirrors the forward rendering kernel's structure (one thread block per tile) but performs the gradient computation. It requires access to many values computed during the forward pass, which must be cached in global memory (e.g., the 2D means, covariances, and per-pixel alpha values).

C++

```
// CUDA Kernel for the backward pass of tile-based rendering
__global__ void render_backward_kernel(
    // Input buffers (from forward pass and loss)
    const float* __restrict__ dL_dImage, //
    const uint64_t* __restrict__ keys, // Sorted keys
```

```

const int* __restrict__ values,      // Sorted Gaussian IDs
const int* __restrict__ tile_ranges, // [num_tiles, 2]
const float* __restrict__ means2d,  // [N, 2]
const float* __restrict__ covs2d,   // [N, 3]
const float* __restrict__ colors,   // [N, 3]
const float* __restrict__ opacities, // [N]
const float* __restrict__ forward_alphas, // - cached alphas

// Output gradient buffers
float* dL_dmeans2d,      // [N, 2] - atomics needed
float* dL_dcovs2d,      // [N, 3] - atomics needed
float* dL_dcolors,      // [N, 3] - atomics needed
float* dL_dopacities,   // [N] - atomics needed

// Parameters
const int image_width,
const int image_height,
const int tile_size)
{
    // --- 1. Identify tile and pixel coordinates (same as forward render) ---
    int tile_x = blockIdx.x;
    int tile_y = blockIdx.y;
    int num_tiles_x = (image_width + tile_size - 1) / tile_size;
    int tile_id = tile_y * num_tiles_x + tile_x;

    int thread_idx = threadIdx.x;
    int pix_in_tile_x = thread_idx % tile_size;
    int pix_in_tile_y = thread_idx / tile_size;
    int pix_x = tile_x * tile_size + pix_in_tile_x;
    int pix_y = tile_y * tile_size + pix_in_tile_y;

    if (pix_x >= image_width |
| pix_y >= image_height) return;

    // --- 2. Load incoming gradient for this pixel ---
    int pixel_offset = (pix_y * image_width + pix_x) * 3;
    float3 dL_dC = {dL_dImage[pixel_offset], dL_dImage[pixel_offset+1],
dL_dImage[pixel_offset+2]};

    // --- 3. Get Gaussian range and initialize backward accumulators ---
    int start_idx = tile_ranges[2 * tile_id];
    int end_idx = tile_ranges[2 * tile_id + 1];
    float3 grad_accumulator = {0.0f, 0.0f, 0.0f};

```

```

// --- 4. Iterate through Gaussians in REVERSE (back-to-front) order ---
for (int i = end_idx - 1; i >= start_idx; --i) {
    int gaussian_id = values[i];

    // Load cached forward alpha for this pixel and Gaussian
    float alpha_prime = forward_alphas[/* index for this pixel, gaussian */];

    // Load Gaussian properties (color, mean, cov, etc.)
    float3 color = {colors[3*gaussian_id], colors[3*gaussian_id+1], colors[3*gaussian_id+2]};
    float2 mean2d = {means2d[2*gaussian_id], means2d[2*gaussian_id+1]};
    //... and so on

    // --- 5. Compute gradients for this Gaussian ---
    float T_i_minus_1 = /* T needs to be recomputed or cached from forward pass */;

    // Gradient w.r.t. color
    float3 dL_dcolor = {
        dL_dC.x * alpha_prime * T_i_minus_1,
        dL_dC.y * alpha_prime * T_i_minus_1,
        dL_dC.z * alpha_prime * T_i_minus_1
    };

    // Gradient w.r.t. alpha_prime
    float dL_dalpha_prime = dot(dL_dC, color) * T_i_minus_1 + dot(grad_accumulator, color);

    // Update the backward accumulator for the next (previous in depth order) Gaussian
    grad_accumulator = grad_accumulator * (1.0f - alpha_prime) - dL_dC * alpha_prime;

    // --- 6. Backpropagate dL/dalpha' to 2D mean and covariance ---
    // d_alpha' = d(opacity * G_2D)
    // dL/d_opacity = dL/d_alpha' * G_2D
    // dL/d_G_2D = dL/d_alpha' * opacity
    // From dL/d_G_2D, compute gradients for mean and covariance using chain rule
    //... (complex derivative calculations omitted for brevity)

    // --- 7. Atomically accumulate gradients to global buffers ---
    // Since a Gaussian can be processed by multiple threads (for different pixels),
    // we must use atomic operations to update its global gradient accumulators.
    atomicAdd(&dL_dcolors[3*gaussian_id], dL_dcolor.x);
    atomicAdd(&dL_dcolors[3*gaussian_id+1], dL_dcolor.y);
    atomicAdd(&dL_dcolors[3*gaussian_id+2], dL_dcolor.z);
    //... atomic adds for dL_dmeans2d, dL_dcovs2d, dL_dopacities
}

```

```
}
```

4.3 Kernel 5: Backpropagation to 3D Parameters

The final kernel in the backward pass takes the gradients with respect to the 2D primitive properties and computes the gradients for the original 3D learnable parameters. This is a mathematically intensive kernel launched with one thread per Gaussian.

C++

```
// CUDA Kernel to backpropagate 2D gradients to 3D Gaussian parameters
__global__ void backprop_kernel(
    // Input buffers (3D params and 2D gradients)
    const float* __restrict__ means3d,
    const float* __restrict__ scales,
    const float* __restrict__ quats,
    const float* __restrict__ view_matrix,
    const float* __restrict__ dL_dmeans2d,
    const float* __restrict__ dL_dcovs2d,
    //... other cached values from forward pass

    // Output gradient buffers
    float* dL_dmeans3d,
    float* dL_dscale,
    float* dL_dquats,

    // Parameters
    const int num_gaussians)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= num_gaussians) return;

    // --- 1. Load 2D gradients for this Gaussian ---
    float2 dL_dmean2d = {dL_dmeans2d[2*idx], dL_dmeans2d[2*idx+1]};
    float3 dL_dcov2d = {dL_dcovs2d[3*idx], dL_dcovs2d[3*idx+1], dL_dcovs2d[3*idx+2]};

    // --- 2. Recompute intermediate values from forward pass ---
    // (e.g., p_cam, Jacobian J, V = W*Sigma*W^T)
```

```

// This is often faster than reading them from global memory if they were cached.

// --- 3. Apply chain rule to get gradients for 3D mean and covariance in camera space ---
// dL/dp_cam = J^T * dL/dmean2d +... (term from covariance gradient)
// dL/dV = J^T * dL/dSigma' * J

// --- 4. Transform gradients from camera space back to world space ---
// dL/dmean_world = W^T * dL/dp_cam
// dL/dSigma = W^T * dL/dV * W

// --- 5. Backpropagate dL/dSigma to gradients for scale and quaternion ---
// This is the most complex part, involving matrix calculus derivatives.
// dL/ds = dL/dSigma : dSigma/ds
// dL/dq = dL/dSigma : dSigma/dq
// where ':' is the Frobenius inner product.

// --- 6. Store final 3D gradients ---
dL_dmeans3d[3*idx] = /*... */;
dL_dscales[3*idx] = /*... */;
dL_dquats[4*idx] = /*... */;
}

```

Section 5: Bridging CUDA and PyTorch: The C++ Bindings

While the core computation is performed in high-performance CUDA kernels, the training loop, optimization, and data loading are managed in Python using the PyTorch framework.¹³ To connect these two worlds, a C++ extension is created that exposes the CUDA rasterizer to Python. This binding layer is responsible for handling data transfer (tensor memory pointers) and for integrating the custom forward and backward passes into PyTorch's automatic differentiation graph.

5.1 The Role of Pybind11 and the PyTorch C++ API

The standard toolchain for this task involves two key components:

- **PyTorch C++ API:** This library provides C++ equivalents of Python's `torch.Tensor` and other utilities. It allows C++ code to manipulate tensors directly on the GPU, access their underlying memory, and define custom operations.
- **Pybind11:** A lightweight, header-only library that creates Python bindings for existing C++ code. It handles the complex details of type conversion, function exposure, and module creation with a clean, modern C++ syntax.

The build process involves compiling the CUDA kernels (.cu files) and the C++ binding code (.cpp file) into a shared library (.so on Linux, .pyd on Windows) that can be imported directly into Python. Users often encounter build issues related to mismatched versions of the CUDA toolkit, the C++ compiler (e.g., GCC or MSVC), and the PyTorch library itself, as documented in numerous community support threads.²⁰ Ensuring a compatible environment is a critical first step.

5.2 Creating a Custom `torch::autograd::Function`

The key to enabling PyTorch's autograd engine to work with custom CUDA code is to subclass `torch::autograd::Function`. This class requires the implementation of two static methods: `forward` and `backward`. PyTorch calls `forward` during the model's forward pass and automatically calls `backward` during backpropagation, passing it the incoming gradients.

Below is a complete, illustrative example of a C++ binding file (ext.cpp) that defines a custom `GaussianRasterization` function and exposes it to Python.

C++

```
#include <torch/extension.h>
#include <vector>

// Forward declarations of our CUDA kernel launchers
// These would be in a separate header file (e.g., "rasterizer.h")
void forward_rasterizer_cuda(/* arguments */);
void backward_rasterizer_cuda(/* arguments */);

// The main custom autograd Function
class GaussianRasterization : public torch::autograd::Function<GaussianRasterization> {
```

```

public:
    // The forward pass function
    static torch::Tensor forward(
        torch::autograd::AutogradContext *ctx,
        // Input Tensors
        const torch::Tensor& means3d,
        const torch::Tensor& scales,
        const torch::Tensor& quats,
        const torch::Tensor& opacities,
        const torch::Tensor& colors,
        const torch::Tensor& view_matrix,
        const torch::Tensor& proj_matrix,
        // Other parameters
        int image_height,
        int image_width
    ){
        // Create the output tensor for the rendered image
        auto opts = torch::TensorOptions().device(means3d.device()).dtype(torch::kFloat32);
        torch::Tensor rendered_image = torch::zeros({image_height, image_width, 3}, opts);

        // --- Call the CUDA forward pass kernels ---
        // This is a simplified call; the real implementation would involve
        // multiple kernel launches and intermediate buffer allocations as
        // described in Section 3.
        forward_rasterizer_cuda(
            means3d.data_ptr<float>(),
            //... pass other tensor data pointers and parameters
            rendered_image.data_ptr<float>()
        );

        // --- Cache necessary tensors for the backward pass ---
        // PyTorch's context object (ctx) is used to store tensors
        // that will be needed during the backward pass.
        ctx->save_for_backward({means3d, scales, quats, opacities, colors, view_matrix});

        return rendered_image;
    }

    // The backward pass function
    static torch::autograd::tensor_list backward(
        torch::autograd::AutogradContext *ctx,
        const torch::autograd::tensor_list& grad_outputs
    ){
        // The incoming gradient from the loss function

```

```

torch::Tensor dL_dImage = grad_outputs;

// Retrieve the tensors saved in the forward pass
auto saved_tensors = ctx->get_saved_variables();
torch::Tensor means3d = saved_tensors;
//... retrieve other saved tensors

// --- Create output tensors for the gradients ---
torch::Tensor dL_dmeans3d = torch::zeros_like(means3d);
torch::Tensor dL_dscale = torch::zeros_like(scales);
//... create other gradient tensors

// --- Call the CUDA backward pass kernels ---
backward_rasterizer_cuda(
    dL_dImage.data_ptr<float>(),
    //... pass saved tensor data pointers
    dL_dmeans3d.data_ptr<float>(),
    dL_dscale.data_ptr<float>()
    //... pass other output gradient pointers
);

// Must return a gradient for each input to the forward function
return {
    dL_dmeans3d,
    dL_dscale,
    //... other gradients
    torch::Tensor(), // Gradient for view_matrix (if not optimized)
    torch::Tensor(), // Gradient for proj_matrix (if not optimized)
    torch::Tensor(), // Gradient for image_height
    torch::Tensor() // Gradient for image_width
};
}
};

// Expose the custom function to Python using pybind11
PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {
    m.def("rasterize_gaussians", &GaussianRasterization::apply, "Differentiable Gaussian
    Rasterizer");
}

```

5.3 Python-side Integration

Once the C++ extension is compiled, it can be used in Python like any other `torch.autograd.Function`. It is common practice to wrap this low-level function call in a more user-friendly `torch.nn.Module`, which makes it feel like a standard PyTorch layer.

Python

```
import torch
import torch.nn as nn
# Import the compiled C++ extension
import diff_gaussian_rasterization_ext as _C

class GaussianRasterizer(nn.Module):
    def __init__(self, image_height, image_width):
        super().__init__()
        self.image_height = image_height
        self.image_width = image_width

    def forward(self, means3d, scales, quats, opacities, colors, view_matrix, proj_matrix):
        # The 'apply' method is automatically generated for the autograd.Function
        return _C.rasterize_gaussians(
            means3d,
            scales,
            quats,
            opacities,
            colors,
            view_matrix,
            proj_matrix,
            self.image_height,
            self.image_width
        )

# --- Example usage within a training loop ---
# rasterizer = GaussianRasterizer(height, width)
# ...
# rendered_image = rasterizer(means3d, scales, quats, opacities, colors, view, proj)
# loss = compute_loss(rendered_image, ground_truth_image)
# loss.backward()
```

```
# optimizer.step()
# optimizer.zero_grad()
```

This seamless integration allows the developer to leverage the extreme performance of custom CUDA kernels while retaining the flexibility and ease of use of the Python-based PyTorch ecosystem for model training and experimentation, as seen in the official train.py scripts.¹³

Section 6: Comparative Analysis and Advanced Optimizations: diff-gaussian-rasterization vs. gsplat

The release of the original 3D Gaussian Splatting paper was accompanied by a reference implementation, diff-gaussian-rasterization, which served as the seminal proof-of-concept and performance benchmark.¹³ In the time since, the community has built upon this foundation, leading to the development of

gsplat, an open-source library that refactors, optimizes, and extends the original rasterizer.²⁵ A comparative analysis of these two codebases reveals the evolution of the technology from a research artifact to a robust, extensible library.

6.1 Architectural Philosophy: Research Code vs. Library

The fundamental difference between the two implementations lies in their design philosophy.

- **diff-gaussian-rasterization:** This is the original code from the authors at Inria.²⁴ It is highly effective and serves as the ground truth for the method's performance and quality. However, its design is monolithic; the CUDA code is tightly coupled to the specific needs of the original paper. It is primarily a research codebase, optimized for a single task and not designed for easy modification or extension.
- **gsplat:** Developed by contributors from the Nerfstudio community, gsplat was explicitly designed to be a modular, developer-friendly, and high-performance backend library.²⁶ Its goal is to serve as a foundational component for a wide range of research projects in Gaussian Splatting.² It features a clean Python API, extensive documentation⁶, and a focus on both performance and usability.

The emergence of gsplat is a natural and important step in the lifecycle of a successful

research idea. It represents the transition from a specialized implementation to a general-purpose tool, enabling broader adoption and faster iteration within the research community. This evolution is a testament to the impact of the original work and the collaborative nature of open-source scientific software development.

6.2 Performance and Memory Optimizations in gsplat

gsplat is not merely a rewrite; it incorporates several significant low-level optimizations that lead to documented improvements in performance and memory efficiency. The library claims up to a 15% reduction in training time and, most notably, up to a 4x reduction in GPU memory footprint compared to the original implementation.⁶ These gains are achieved through several advanced CUDA programming techniques.

- **Memory Layout and Coalescing:** The packed mode offered by gsplat's rasterization function likely refers to an optimized memory layout for the Gaussian parameters.¹⁸ Instead of storing each attribute (positions, scales, rotations, etc.) in a separate, large array (an "Array of Structures" or AoS approach), a "Structure of Arrays" (SoA) layout can be more efficient. However, within the kernels, packing related data together can improve memory coalescing. For instance, loading (x, y, z, opacity) for a Gaussian in a single 128-bit transaction is more efficient than four separate 32-bit transactions. gsplat's optimizations likely involve careful data layout choices to maximize the bandwidth of global memory reads and writes.
- **Sparse Gradients:** One of the most significant memory optimizations is the option for `sparse_grad`.¹⁸ In the standard backward pass, dense gradient tensors are created for all parameters of all Gaussians, even for Gaussians that were culled or had a negligible contribution to the final image. This is wasteful. The sparse gradient approach changes the output of the backward pass. Instead of dense tensors, it produces gradients in a sparse format, such as Coordinate list (COO), which stores (index, value) pairs only for the Gaussians that received a non-zero gradient. This dramatically reduces the memory required for storing gradients, especially in large scenes where many Gaussians may be outside the view frustum at any given time.
- **Kernel Fusion and Register Usage:** While not explicitly documented, performance gains in CUDA often come from kernel fusion—combining multiple, small kernel launches into a single, larger one to reduce launch overhead—and careful management of register usage. gsplat's developers, with a focus on performance, have likely hand-tuned the CUDA kernels to minimize register spilling (writing registers to local memory, which is slow) and to maximize occupancy (the number of active warps per SM).

6.3 Advanced Rendering Features

Beyond raw performance, gsplat introduces a suite of advanced features not present in the original implementation, reflecting its role as a versatile library for ongoing research.²

- **N-Dimensional Rasterization:** The rasterizer can be configured to render arbitrary-dimensional feature vectors instead of just 3-channel RGB color. This is crucial for research that combines Gaussian Splatting with learned feature fields, where each Gaussian might carry a high-dimensional feature vector that is later interpreted by a small neural network.² This is implemented by templating the CUDA kernels on the feature dimension and modifying the blending loop to accumulate N-dimensional vectors.
- **Depth Rendering:** The library provides built-in support for rendering depth maps, offering modes for both accumulated z-depth and expected depth.¹⁸ This is a vital feature for applications in 3D reconstruction, robotics, and for using depth-based loss functions during training to regularize the geometry of the scene.
- **Anti-Aliasing:** gsplat implements the anti-aliasing technique proposed in subsequent research.²⁷ When viewing a scene from a distance or at a lower resolution, aliasing artifacts can appear as flickering or shimmering. The anti-aliasing mode mitigates this by ensuring that the projected 2D splats do not become smaller than a certain size (e.g., half a pixel), effectively pre-filtering the representation to match the sampling rate of the image grid.

The following table summarizes the key differences between the two implementations, providing a clear rationale for developers choosing a backend or seeking to implement advanced features.

Feature/Metric	diff-gaussian-rasterization (Original)	gsplat (Library)	Implementation Notes
Performance	Baseline (High)	Up to 15% faster training time	Achieved through kernel tuning, reduced overhead, and optimized memory access patterns.
Memory Usage	Baseline (High)	Up to 4x less VRAM	Primarily due to the sparse gradient implementation, which avoids

			allocating dense tensors for all parameters.
Modularity	Monolithic, research-specific	Modular, API-driven library	gsplat is designed to be imported and used as a backend component in other projects like Nerfstudio.
N-D Features	No	Yes	CUDA kernels are templated to support blending of arbitrary-dimensional feature vectors.
Depth Rendering	No	Yes	The forward rendering kernel has modes to output accumulated or expected depth maps.
Anti-Aliasing	No	Yes	Implements a technique to clamp the minimum size of 2D splats to prevent aliasing.
Sparse Gradients	No	Yes	The backward pass can output gradients in a memory-efficient COO sparse format.

Section 7: Conclusion and Future Directions

The advent of 3D Gaussian Splatting has marked a significant milestone in computer graphics, achieving an unprecedented combination of photorealistic rendering quality and real-time performance. The key to this success lies not only in the novel scene representation but equally in the highly specialized, differentiable GPU rasterization pipeline. This report has provided an exhaustive technical deep dive into the design and implementation of this pipeline, from its mathematical underpinnings to the practicalities of writing and optimizing the custom CUDA kernels that drive it.

7.1 Summary of Best Practices

The analysis and code examples presented herein distill several core principles for any developer seeking to implement or extend a high-performance 3DGS rasterizer:

1. **Embrace the Tile-Based Architecture:** The division of the screen into tiles is the foundational architectural decision that enables massive parallelism and efficient memory usage on the GPU. Any high-performance implementation must be built around this concept.
2. **Leverage Parallel Sorting:** The use of a device-wide, high-performance radix sort is the lynchpin that makes the tile-based approach feasible. It efficiently solves the complex problem of ordering millions of transparent primitives, transforming the final rendering stage into an embarrassingly parallel task. The performance of this sort is often the primary bottleneck and a key area for optimization.
3. **Optimize Memory Access:** Performance is dictated by memory bandwidth. Techniques such as using shared memory to cache data within a tile, choosing optimal memory layouts (e.g., packed structures), and minimizing global memory traffic are paramount.
4. **Integrate via C++ Extensions:** For use in modern deep learning frameworks, a C++ binding layer that integrates seamlessly with PyTorch's autograd engine is essential. This allows the high-performance CUDA backend to be driven by flexible and powerful Python-based training scripts.

7.2 Emerging Research and Future Work

The field of 3D Gaussian Splatting is evolving at a rapid pace, with active research addressing

the method's current limitations and expanding its capabilities. The foundational rasterizer described in this report serves as a platform for these innovations.

- **Algorithmic Simplification and Speed:** Recent work has explored the use of simpler primitives, such as **isotropic Gaussians**, which are constrained to be spheres rather than arbitrary ellipsoids.²⁸ While this may reduce the ability to represent fine geometric details with a single primitive, it significantly simplifies the projection and covariance calculations, potentially leading to even faster rendering speeds.
- **Improved Physical Accuracy:** The standard 3DGS rasterizer makes approximations for the sake of speed. New research on **Volumetrically Consistent Rasterization** aims to improve physical accuracy by deriving the alpha values from an analytical calculation of transmittance through the Gaussian volume.²⁹ This approach promises to more faithfully model the volume rendering equation, potentially leading to higher-quality reconstructions, particularly for opaque surfaces, while retaining the speed benefits of rasterization.
- **Memory Footprint Reduction:** A primary drawback of 3DGS is the large memory footprint of the models, which can consist of millions of Gaussians, each with dozens of parameters. Active research is focused on **compression and quantization** techniques to reduce this storage requirement.³⁰ Methods like Compact3D use vector quantization to represent Gaussian attributes with a shared codebook, significantly reducing the per-Gaussian data size and making the models more practical for deployment on memory-constrained devices.

These research frontiers demonstrate that while the core principles of differentiable, tile-based rasterization are now well-established, there remains ample room for innovation. Future work will undoubtedly continue to push the boundaries of speed, quality, and efficiency, further solidifying the role of splatting-based methods as a cornerstone of real-time radiance field rendering.

Works cited

1. gsplat: An Open-Source Library for Gaussian Splatting - arXiv, accessed September 6, 2025, <https://arxiv.org/html/2409.06765v1>
2. gsplat documentation, accessed September 6, 2025, <https://docs.gsplat.studio/>
3. 3D Gaussian Splatting for Real-Time Radiance Field Rendering - Inria, accessed September 6, 2025, <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>
4. Introduction to 3D Gaussian Splatting - Hugging Face, accessed September 6, 2025, <https://huggingface.co/blog/gaussian-splatting>
5. 3D Gaussian Splatting Introduction – Paper Explanation & Training on Custom Datasets with NeRF Studio Gsplats - LearnOpenCV, accessed September 6, 2025, <https://learnopencv.com/3d-gaussian-splatting/>
6. Original reference implementation of "3D Gaussian Splatting for Real-Time Radiance Field Rendering" - GitHub, accessed September 6, 2025, <https://github.com/graphdeco-inria/gaussian-splatting>

7. 3D Gaussian Splatting for Real-Time Radiance Field Rendering - Inria, accessed September 6, 2025, https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/3d_gaussian_splatting_low.pdf
8. A Comprehensive Overview of Gaussian Splatting - Towards Data Science, accessed September 6, 2025, <https://towardsdatascience.com/a-comprehensive-overview-of-gaussian-splatting-e7d570081362/>
9. 3D Gaussian Splatting for Real-Time Radiance Field Rendering - ResearchGate, accessed September 6, 2025, https://www.researchgate.net/publication/372989904_3D_Gaussian_Splatting_for_Real-Time_Radiance_Field_Rendering
10. Rasterization - gsplat documentation, accessed September 6, 2025, <https://docs.gsplat.studio/main/apis/rasterization.html>
11. Things April 2024: Gaussian splatting, Metal vs. CUDA - Andrew Chan, accessed September 6, 2025, <https://andrewkchan.dev/posts/things-april-2024.html>
12. Unable to install the submodules with pip · Issue #535 · graphdeco-inria/gaussian-splatting, accessed September 6, 2025, <https://github.com/graphdeco-inria/gaussian-splatting/issues/535>
13. Error installation of diff-gaussian-rasterization and found solution · Issue #332 - GitHub, accessed September 6, 2025, <https://github.com/graphdeco-inria/gaussian-splatting/issues/332>
14. gsplat fails to build with gaussian splatting pipeline · Issue #2685 - GitHub, accessed September 6, 2025, <https://github.com/nerfstudio-project/nerfstudio/issues/2685>
15. Original reference implementation of "3D Gaussian Splatting for Real-Time Radiance Field Rendering" - GitHub, accessed September 6, 2025, <https://github.com/bycloud-AI/gaussian-splatting-Windows>
16. GraphDeco - INRIA - GitHub, accessed September 6, 2025, <https://github.com/graphdeco-inria>
17. nerfstudio-project/gsplat: CUDA accelerated rasterization of gaussian splatting - GitHub, accessed September 6, 2025, <https://github.com/nerfstudio-project/gsplat>
18. Splatfacto - nerfstudio, accessed September 6, 2025, <https://docs.nerf.studio/nerfology/methods/splat.html>
19. gsplat: An Open-Source Library for Gaussian Splatting - Journal of Machine Learning Research, accessed September 6, 2025, <https://www.jmlr.org/papers/volume26/24-1476/24-1476.pdf>
20. [2403.14244] Isotropic Gaussian Splatting for Real-Time Radiance Field Rendering - arXiv, accessed September 6, 2025, <https://arxiv.org/abs/2403.14244>
21. [2412.03378] Volumetrically Consistent 3D Gaussian Rasterization - arXiv, accessed September 6, 2025, <https://arxiv.org/abs/2412.03378>
22. 3D Gaussian Splatting In Practice, accessed September 6, 2025, https://3dgsutorial.github.io/3dv_part2.pdf